

Київський національний університет імені Тараса Шевченка

Факультет комп'ютерних наук та кібернетики

Кафедра інтелектуальних програмних систем

ЗВІТ

з виробничої практики з відривом від навчання

з освітньо-професійної програми «Програмна інженерія»

за спеціальністю 121 Інженерія програмного забезпечення

Виконав

Студент 3-го курсу

Тесленко Назар Олександрович

Керівник практики від кафедри:

доцент, кандидат фіз.-мат. наук

Супрун Олег Олексійович

Керівник практики від бази
практики:

ФОП Левківський Євгеній

Сергійович

Засвідчую, що в цій роботі
немає запозичень з праць
інших авторів без відповідних
посилань

Студент Тесленко Назар Олександрович

ЗМІСТ

Вступ.....	3
1. Постановка задачі.....	4
2. Процес реалізації.....	4
Архітектура бази даних та модуля	5
Побудова ETL-модуля.....	6
3. Опис функціональності.....	7
Екстракція даних з Meta Graph API	7
Мережеві обмеження (Rate Limiting) та стійкість до відмов.....	7
Валідація та нормалізація даних.....	8
4. Тестування, обробка помилок та рефакторинг.....	9
5. Результат практики	10
Висновок.....	12

Вступ

Під час проходження виробничої практики основним завданням була розробка бекенд-модуля для автоматизації збору маркетингової аналітики. Основною бізнес-проблемою була необхідність мануального отримання основних метрик та другорядних даних з рекламних кабінетів Meta, для проведення подальшого аналізу. Це вимагало значних часових витрат та створювало ризик помилок через людський фактор.

Метою роботи стала повна автоматизація цього процесу шляхом розробки надійного ETL-пайплайну (Extract, Transform, Load). Архітектурне рішення передбачає три чітко розподілені етапи: екстракція «сирих» даних через Meta Graph API, їх трансформація(очищення, приведення типів, обробка відсутніх значень) та завантаження до бази даних.

Для реалізації модулів обробки даних використовувалася мова Python із застосуванням функціональної парадигми програмування. Цей підхід дозволяє розбити складний процес на незалежні, ізольовані функції, що спрощує кінцеве тестування кожного компоненту окремо та забезпечити зручність розширення модуля в майбутньому.

1. Постановка задачі

Мета:

Розробка серверного модуля екстракції даних (Data Extraction Module) для автоматизованого збору даних з рекламних кабінетів через Meta Graph API та їх збереження до бази даних.

Основні завдання:

- Розробка бекенд модулів для подальшого виконання запитів до Meta Graph API з метою отримання ключових рекламних метрик: reach, impressions, clicks, spend, cpc, cpm, ctr, conversions та інших показників ефективності рекламних кампаній
- Обробка отриманих відповідей від API у форматі JSON, розбір багаторівневих структур даних та виокремлення необхідної інформації
- Проектування схеми бази даних PostgreSQL для збереження отриманих метрик із необхідною прив'язкою до дати та часу. Налаштування зв'язків між новими таблицями та існуючими таблицями клієнтів і кампаній для забезпечення цілісності даних.
- Впровадження стійких до відмов механізмів: обробка мережевого таймауту, ліміти запитів API (Too many requests) та валідація некоректних типів даних

Реалізація є виключно серверним сервісом обробки даних без графічного інтерфейсу.

2. Процес реалізації

Використані технології та інструменти:

Python – основна мова програмування. Реалізація побудована за використання функціональної парадигми з метою розбити логіку на ізольовані модулі.

requests – бібліотека для реалізації синхронних http/https запитів до Rest API Meta.

PostgreSQL – реляційна база даних для збереження зібраної аналітики, що забезпечує жорстку типізацію даних та підтримку складних зв'язків між таблицями.

psycopg2 – PostgreSQL адаптер для пайтону для імплементації та роботи з базою даних.

json, datetime модулі – для приведення даних отриманих від сервера та форматування часових позначок відповідно до схеми таблиць.

Архітектура бази даних та модуля

Для уникнення дублювання інформації та дотримання третьої нормальної форми, було спроектовано та створено наступні таблиці:

- Clients (existed entity)
 - Зберігає відповідну інформацію про замовників, є первинною сутністю в ієрархії зв'язків.
- Ad_accounts
 - Зберігає інформацію про рекламні кабінети та відповідні їм дані. Кожен кабінет належить конкретному клієнту через зовнішній ключ client_id.
- Ad_campaign
 - Містить ідентифікаційні поля кампаній (campaign_id), їх назви, статус активності, client_id(foreign key), що вказує на власника кампанії.
- Ad_data (main entity)
 - Головна таблиця для збереження метрик
 - Поля: id (PRIMARY KEY); campaign_id (FOREIGN KEY); report_date (TIMESTAMP); reach, impressions, clicks (INTEGER), cps, cpm, spend (NUMERIC).
 - Для підтримки цілісності, зв'язок з Ad_campaign налаштований через ON DELETE CASCADE

Така структура дозволяє правильно визначити належність набору наших метрик до конкретної кампанії, кабінету та клієнту, що матиме вагомий вплив у подальшому аналізі даних.

Побудова ETL-модуля

Архітектуру застосунку було розділено на три логічні ізольовані модулі, кожен з яких виконує відповідно визначене завдання (Single Responsibility)

- **Data Extraction** - звернення до API платформи Meta. Виконується Http-запит за яким отримуємо неструктуровані дані у JSON-форматі разом із заголовками, що містять інформацію про поточний стан лімітів.
- **Data Transformation** - проміжний шар обробки даних. Приймає отримані дані з API, фільтрує надлишкові поля, виконує типізацію текстових метрик відповідно до типу їх поля у базі даних (integer, numeric), а також обробляє пропущені значення (Null). На виході маємо структурований словник даних, для відповідної кампанії
- **Data Loading** – приймає приведені до правильного вигляду метрики, і після з'єднання з базою даних формує SQL-запит для безпечного запису інформації до таблиці Ad_data у межах однієї транзакції.

Важливо окреслити, що у процесі модулі взаємодіють між собою послідовно, тобто вихідні дані одного є вхідними даними наступного. Це забезпечує чітке розмежування відповідальності та дозволяє тестувати і розширювати кожен модуль незалежно від інших.

3. Опис функціональності

Розроблений модуль працює як автономний сервіс обробки даних. Його головна задача – отримання та трансформація інформації від стороннього сервера до локальної бази даних. Логіка роботи поділена на кілька ключових функціональних блоків, кожен з яких вирішує свої специфічні проблеми.

Екстракція даних з Meta Graph API

Процес отримання даних починається з формування Http GET-запитів до endpoint-ів Meta API. Задля дотримання вимог безпеки, для автентифікації запитів використовується протокол надання доступу за токенами (Access Tokens). У headers кожного запиту передається згенерований Bearer Token, що гарантує доступ до даних рекламних кабінетів лише авторизованому сервісу.

Однією з особливостей роботи з Meta API є механізм пагінації. При запиті великих обсягів даних – наприклад статистика за тривалий період по великій кількості кампаній – сервер не повертає усі записи одразу, а розбиває відповідь на сторінки з обмеженою кількістю записів (HTTP-пакети).

Для вирішення цієї задачі було реалізовано логіку курсорної пагінації (Cursor-based Pagination). Функція витягування даних зчитує параметр next з об'єкта paging у JSON-відповіді та рекурсивно формує новий запит, доки не буде вивантажено повний масив даних за вказаний проміжок часу. Це дозволяє коректно обробити рекламні кабінети будь-якого розміру незалежно від кількості кампаній та часового діапазону запиту.

Мережеві обмеження (Rate Limiting) та стійкість до відмов

Під час реалізації модулів була виявлена проблема: при спробі вивантаження великого об'єму даних (статистика за тривалий період, по великій кількості кампаній), сервера починають блокувати з'єднання, повертаючи помилку «429 Too many requests», що є базовим механізмом захисту від перенавантажень.

Rate Limiting згідно з документацією:

Використовується система динамічних лімітів – **Business UC Rate Limiting**, що допомагає серверам уникати несправедливого розподілення ресурсів між запитами різних клієнтів.

В кожній успішній відповіді у заголовках надсилається X-Business-Use-Case-Usage, де у форматі JSON представлені наступні цифри:

Приклад: {"call_count": 75, "total_time": 60, "total_cputime": 62}, де

call_count – відсоток використання запитів

total_time – відсоток використання часу сервера

Якщо один з критеріїв досягає 100, у відповідь на запит повертається помилка 429 «Too many requests»

Для вирішення даної проблеми була реалізована наступна логіка. Після кожного успішного запиту модуль перевіряє вище згадані параметри у заголовку відповіді. Якщо хоча б один з параметрів перевищує ланку у 80%, виконуємо наступні дії:

- Зберігаємо в оперативній пам'яті маркер наступної сторінки – посилання next з об'єкта paging
- Робимо таймаут у 300 секунд
- По закінченню таймауту, робимо повторний запит передаючи до запиту раніше збережене посилання next. Це дозволяє уникнути дублювання інформації, та продовжити збір даних з місця зупинки.
- З отриманням першої відповіді після паузи зчитуємо заголовок X-Business-Use-Case-Usage. Оскільки ліміти є динамічними то за період паузи їх відсоткові значення знижуються до задовільних, після чого модуль автоматично повертається до попередньої швидкості виконання запитів.

Такий підхід забезпечує self-healing поведінку модуля – здатність самостійно відновлювати роботу без втрати прогресу та без ручного втручання.

Валідація та нормалізація даних

Після успішного отримання даних від Meta Graph Api виникли певні питання щодо їхньої обробки. Відповіді від сервера мають багаторівневу JSON-структуру з вкладеними списками та словниками, при цьому дані за певних причин можуть бути не повними.

- Spend (поля фінансових витрат) можуть бути відсутніми у JSON-відповіді, якщо рекламна кампанія була на паузі в конкретний день.
- Деякі числові метрики повертаються у вигляді текстового рядку замість числового типу.

Прямий запис таких даних у базу даних призведе до падіння транзакцій через конфлікт типів даних або порушенням обмежень NOT NULL.

Для вирішення цих проблем було створено модуль нормалізації даних.

- Функція обходить багаторівневий JSON-об'єкт та витягує цільові метрики, формуючи результатний словник.
- Використовується метод безпечного доступу до ключів словника, тобто якщо очікувана метрика відсутня, їй автоматично присвоюється значення за замовчуванням (0 – для числових значень)
- Необхідним етапом також було кастування типів отриманих даних. Текстові значення фінансових витрат, кількості – числові показники – примусово конвертуються до float або integer відповідно до схеми таблиці Ad_data у базі даних.

В результаті на вхід модуля завантаження завжди надходять коректно типізовані та структуровані дані, що гарантує стабільні транзакції незалежно від якості вхідних даних від API.

4. Тестування, обробка помилок та рефакторинг

Після реалізації основних модулів було проведене тестування для відзначення коректності їх роботи. В процесі тест-ранів виявились кілька неочевидних проблем дрібних та більш важливих проблем, серед яких:

- Забезпечення ідемпотентності записів до БД

При повторних запусках модуля при мережевому збої виникали конфлікти унікальності, оскільки система намагалася вставити дані, які вже частково були завантажені.

Для вирішення даної проблеми було переписано SQL-запити з використанням конструкції «ON CONFLICT (...) DO UPDATE», що гарантує коректне оновлення існуючого запису замість створення дублікатів.

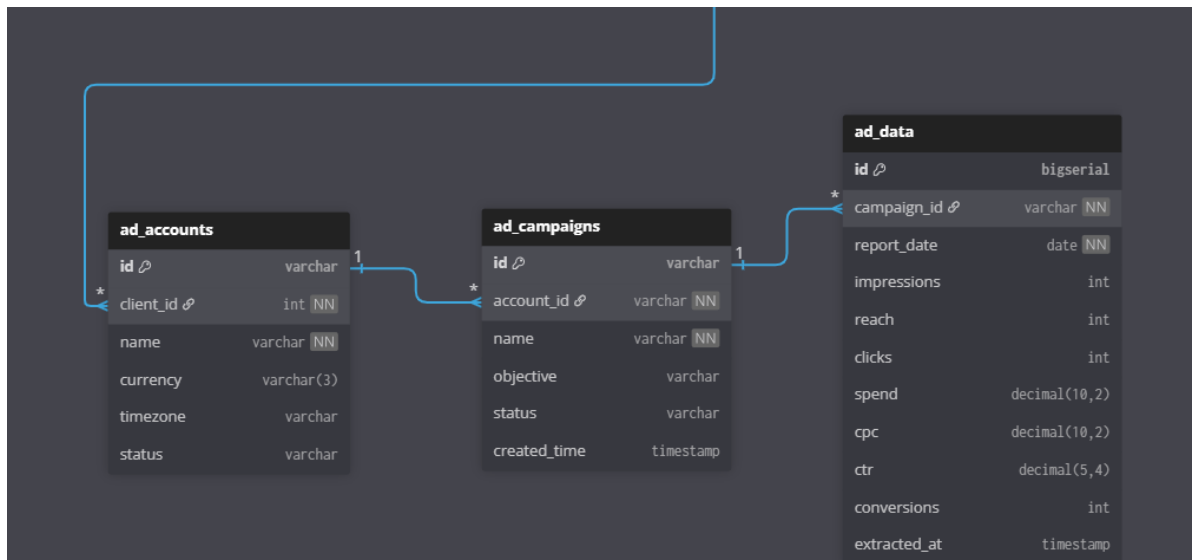
Спираючись на результати тестування були виправлені дрібні помилки та проведено невеликий рефакторинг, за якого були винесені повторні фрагменти логіки для зменшення дублювання коду.

5. Результат практики

Результатом виконання практичного завдання є повноцінний ETL-пайплайн, який автоматизує процес збору маркетингової аналітики. Оскільки розроблений модуль є бекенд-сервісом без графічного інтерфейсу, візуальним результатом роботи є спроектована схема бази даних та архітектурна схема модулів обробки даних.

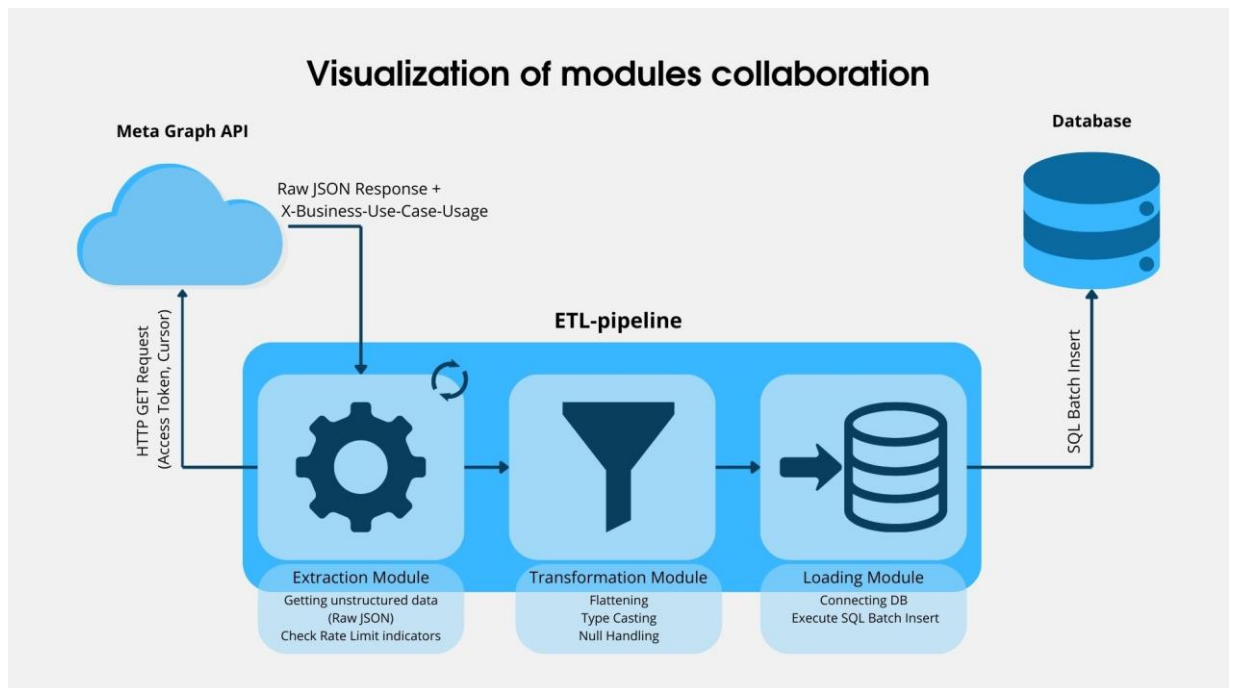
Загальна ER-модель відображає чотири таблиці та зв'язки між ними: Clients як коренева сутність, Ad_accounts з прив'язкою до клієнта, Ad_campaign з прив'язкою до кабінету та Ad_data як головна таблиця метрик. Реалізована структура забезпечує нормалізацію даних до третьої форми та унеможливлює дублювання інформації.

ER-модель доданих таблиць до існуючої структури бази даних:

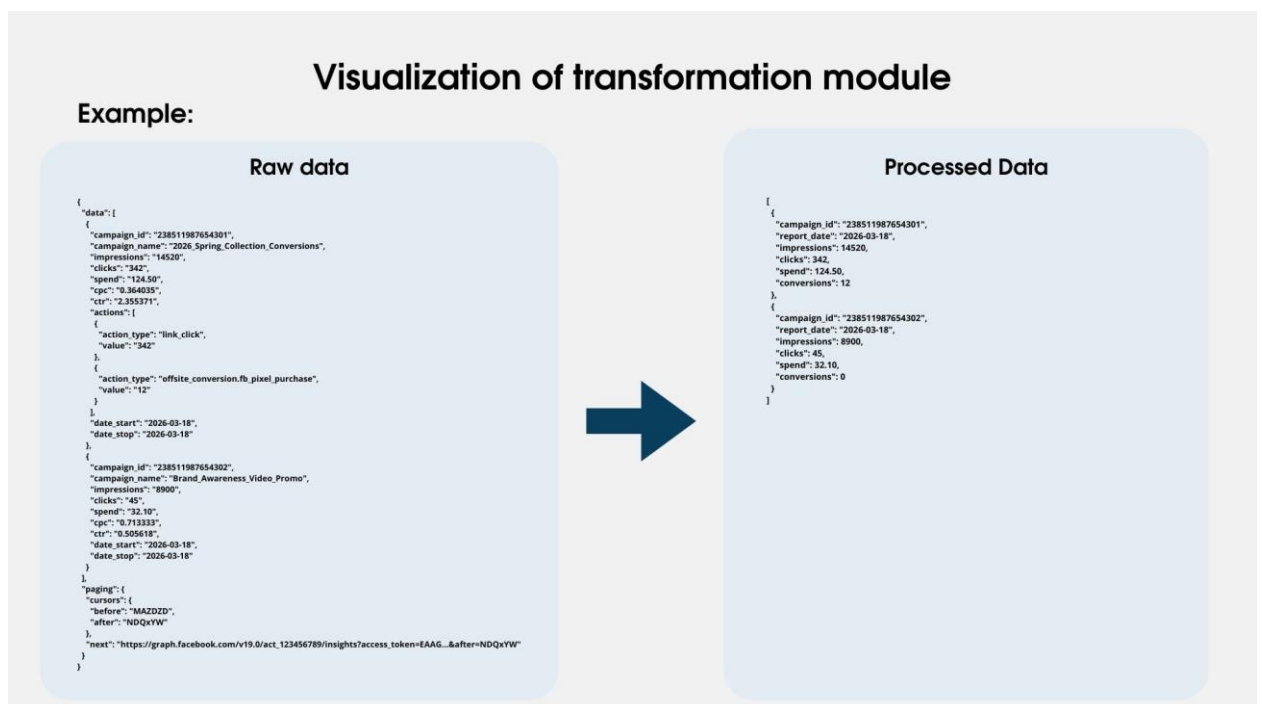


Архітектурна схема взаємодії модулів ілюструє повний цикл обробки даних від HTTP запиту, з повною обробкою через пайплайн, до фінального SQL Batch Insert до бази даних.

Для розуміння загального алгоритму роботи, було побудовано візуальну схему взаємодії програмних модулів:



Візуалізація модуля трансформації демонструє різницю між вхідними та вихідними даними. Зліва, скорочений вигляд сирих даних отриманих від Meta API, натомість справ можемо бачити вигляд оброблених даних з плоскою структурою, коректними типами та з усіма заповненими значеннями, готових до подальшого використання.



Висновок

Під час проходження виробничої практики було розроблено повноцінний бекенд-модуль для автоматизованого збору та обробки маркетингової аналітики рекламних кабінетів Meta. Звертаючись до календарного плану, усі поставлені завдання були виконані.

У процесі було практично застосовано навички роботи із REST API, проєктування нормалізованих баз даних, побудови модульної архітектури з дотриманням Single Responsibility принципу. Окрім цього, набутий новий досвід самостійного дослідження роботи та реалізації нетривіальних механізмів: курсорна пагінація та динамічний Rate Limiting.

Побудований комплекс модулів є готовою основою для подальших інтеграцій. Одним з прикладів може бути інтеграція AI-шару для автоматичного виявлення аномалій у метриках, прогнозування ефективності кампаній або генерації аналітичних звітів. Використання реалізованого пайплайну з інструментами штучного інтелекту надає можливість створити повноцінну систему інтелектуальної аналітики рекламних даних.